

第6讲

训练集、验证集、测试集、过拟合与泛化

为什么“训练得很好”不等于“真的学会了”？

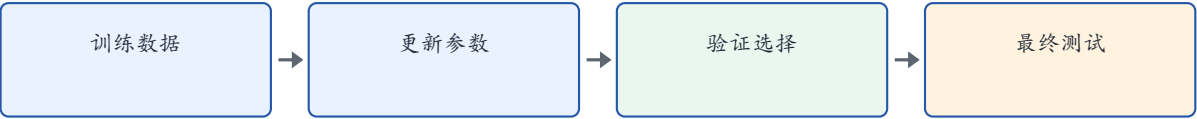
本讲目标

学完后，你应该能解释：训练集、验证集、测试集各自负责什么；什么叫过拟合；什么叫泛化；为什么测试集不能反复偷看；以及这些概念为什么对 LLM 与 Agent Systems 的科研评测尤其关键。

先记住一句话

一句话心智模型

训练集像“平时练习题”，验证集像“模拟考试”，测试集像“最后一次真正考试”。模型不是为了把练习题背下来，而是为了在没见过的新题上仍然表现好。



1. 为什么我们需要把数据分开？

上一讲你已经知道，模型会通过 Loss、反向传播和优化器不断修改参数。如果我们把所有数据都用来训练，然后再拿这些同样的数据评估，结果会产生一个严重问题：模型可能只是“记住了答案”，而不是学到了能推广到新数据上的规律。

一份数据集被“分角色”使用，而不是三个名字而已

训练集 Train 70%	验证集 Val 15%	测试集 Test 15%
---------------	-------------	--------------

训练集：改参数 验证集：选方案/调超参数 测试集：最后一次公正考试

最基本的数据划分：

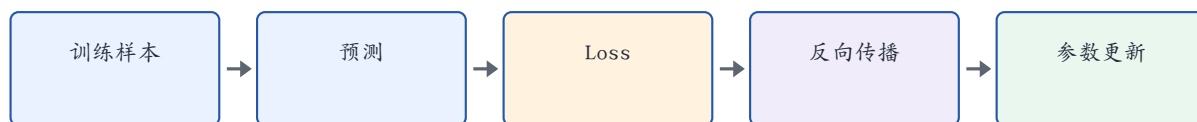
集合	主要用途	能否更新模型参数？	你可以把它理解成
训练集 Training Set	计算 Loss、反向传播、更新参数	可以	平时做题
验证集 Validation Set	选模型、调超参数、早停、比较方案	通常不可以	模拟考试
测试集 Test Set	训练结束后做最终客观评估	不可以	正式考试

关键区别

训练集直接“塑造”模型参数；验证集虽然不更新参数，但会影响你做什么选择；测试集应该尽量保持独立，只在最终报告结果时使用。

2. 训练集：模型真正“吃进去”的数据

训练时，一条样本会经过：前向计算 → 得到预测 → 计算 Loss → 反向传播 → 得到梯度 → 优化器修改参数。这个过程会反复发生很多次。



因此，训练集对模型的影响是最直接的。一个样本如果进入训练集，它就可能改变参数。

对于语言模型，训练样本往往不是传统的“图片+标签”，而是大规模文本序列。模型的任务是根据前面的 token 预测下一个 token。

例如：

输入上下文: **The capital of France is**

目标 token: **Paris**

如果模型给 Paris 的概率很低，Loss 就会较高，参数会被调整，使类似上下文下 Paris 的概率上升。

注意

“训练集里的知识进入模型”不是把文本原样复制到一个数据库里。更准确地说，是训练数据通过梯度更新，改变了大量参数。模型可能学到统计规律，也可能记忆部分具体内容。

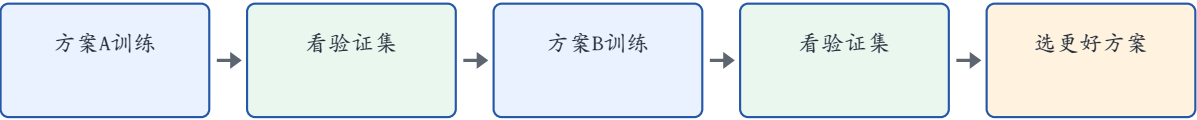
3. 验证集：为什么它不是“多余的一份测试集”？

模型训练过程中，你需要做很多选择，但这些选择本身也可能导致过拟合。例如：学习率选多少？训练多少轮？网络多大？用哪一种提示模板？Agent 要不要加反思模块？

这些不是神经网络权重本身，而是超参数（Hyperparameters）或实验设计选择。通常我们利用验证集来判断这些选择哪一个更好。

参数 Parameter	超参数 Hyperparameter
模型通过梯度自动学到	研究者/训练流程决定
例如权重 w 、偏置 b	例如学习率、Batch Size、训练轮数
训练集直接更新	通常根据验证集表现选择

一个典型流程：



重要

虽然验证集不参与反向传播，但如果你反复根据验证集结果修改方案，它实际上也在“影响”你的最终系统。因此验证集也可能被“过拟合”。

4. 测试集：为什么不能反复偷看？

假设你每做完一次模型改动，就去看测试集表现，然后根据测试集结果再调整模型。虽然测试数据没有直接进入反向传播，但你已经通过人的决策把测试信息反馈进了研究过程。

这叫做测试集泄漏（Test Leakage）或更广义的评测污染。结果是：测试集不再代表真正未知的数据。

考试比喻

如果老师提前把正式考试卷给你看，然后你根据错题重新复习，再拿同一张卷子考试，这个分数当然不能再代表真实水平。

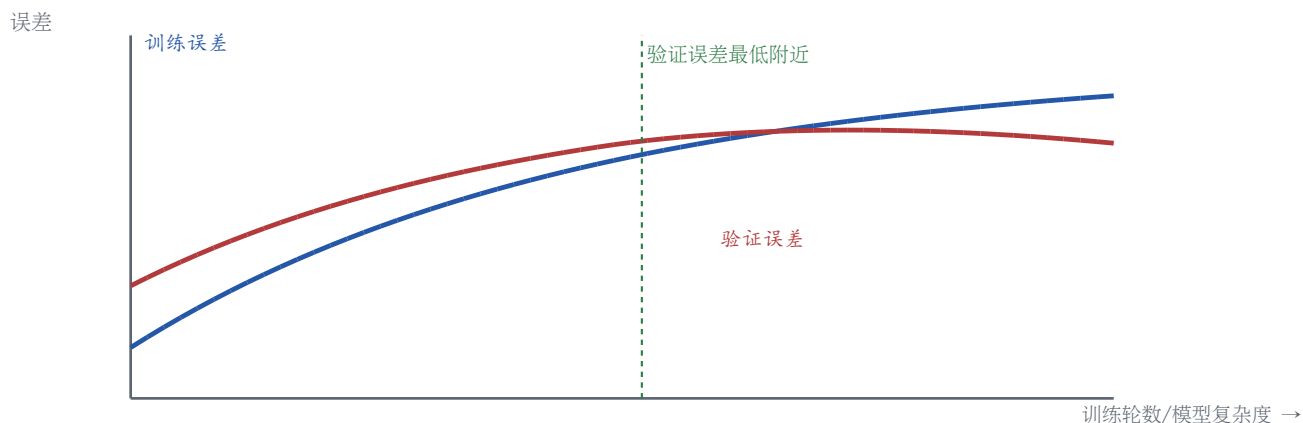
科研里最常见的错误之一，就是：

- 反复跑测试集，直到结果好看；
- 看到某个 benchmark 分数后，专门改 prompt、规则、工具或数据，让它在这个 benchmark 上更高；
- 论文之间长期使用同一个公开 benchmark，社区不断根据排行榜优化，最后 benchmark 本身被“集体过拟合”。

这在今天的 LLM 与 Agent 评测中特别重要，因为很多 benchmark 是公开的，题目、答案甚至评测脚本都能被研究者反复查看。

5. 什么叫过拟合 (Overfitting) ?

过拟合不是“模型太聪明”，而是模型对训练数据中的细节、偶然噪声、特殊模式拟合得过头，以至于面对新数据时反而变差。



图中的典型现象：训练误差继续下降，但验证误差先下降、随后又上升。这个分叉就是过拟合的重要信号。

通俗例子：

某学生做了 100

道模拟题。如果他真正理解了“二次方程”的规律，那么换一批新题仍然会做；如果他只是把 100 道题的答案背了下来，训练题能拿满分，但新题会迅速暴露问题。

不要误解

“训练误差低”本身不是坏事。问题在于：训练表现很好，而新数据表现明显差。过拟合是训练表现和泛化表现之间的落差。

6. 什么叫欠拟合 (Underfitting) ?

欠拟合是另一个方向的问题：模型连训练数据里的基本规律都没有学好。

状态	训练表现	验证/测试表现	典型原因
欠拟合	差	差	模型太弱、训练不足、特征不足、优化失败
合理拟合	好	也好	学到了可泛化规律
过拟合	非常好	明显变差	模型过度记忆训练细节/噪声

你可以把模型能力想成一张网：

太粗的网抓不住规律 → 欠拟合；

网过于贴合每一个训练样本的细小波动 → 过拟合；

真正理想的是抓住稳定规律，而不是每一个偶然细节。

科研判断

如果训练集和验证集都表现差，首先怀疑“没学会”；如果训练集很好、验证集明显差，首先怀疑“过拟合或分布差异”。

7. 泛化 (Generalization) 到底是什么？

泛化是机器学习最核心的目标之一：模型在没有见过的新样本、新组合或新条件下，仍然能保持合理表现。



泛化真正关心：模型能否在“没见过但相关”的条件下仍然可靠

但“新数据”不是一个单一概念。泛化至少可以分为：

类型	含义	例子
同分布泛化 IID	新样本来自与训练数据近似相同的分布	同类型数学题，但题目不同
分布外泛化 OOD	新样本与训练分布发生明显变化	训练是短问题，部署遇到超长上下文
组合泛化	已见过的元素以新组合出现	已学工具A/B，但第一次需要复杂组合调用
时间泛化	环境随时间变化	软件版本、网页结构、知识更新

对于 Agent Systems，真正困难的往往不是 IID 泛化，而是环境变化、工具失败、上下文积累、其他 Agent 行为变化等造成的分布漂移。

8. 一个最小数学直觉：我们真正关心的是“未知总体误差”

训练时，我们只能看到有限样本。真正希望优化的却是模型在未来所有可能样本上的平均表现。

经验风险 (Empirical Risk)：在手头训练样本上的平均 Loss。

期望风险 (Expected Risk)：在真实数据分布上的平均 Loss。

机器学习的难点就在于：我们只能直接计算前者，却希望后者也小。

核心矛盾

训练集是有限的，真实世界几乎无限。泛化研究的本质，就是研究：为什么在有限样本上学到的参数，能够对未见数据仍然有效？

这也是为什么“训练集分数”并不能回答我们真正关心的问题。

9.

为什么模型越大，不一定越容易简单地判断过拟合？

在传统教材里，经常用“模型太复杂 → 更容易过拟合”作为直觉。这在很多场景有用，但现代深度学习和大模型更复杂。

- 大模型参数很多，但训练数据也可能极其庞大；
- 优化器、正则化、数据增强、预训练结构会改变泛化行为；
- 同一个模型既可能记忆某些训练样本，也能在很多任务上产生强泛化；
- 参数数量本身不能单独决定是否过拟合。

因此，不要把“参数多”简单等同于“过拟合”。判断还是要看：训练分布与新分布上的表现差异，以及是否存在数据泄漏、记忆、污染等证据。

对 LLM 特别重要

LLM 可能同时具有“广泛规律学习”和“局部训练样本记忆”。这两件事可以同时发生，并不矛盾。

10. 数据泄漏 (Data Leakage) 到底有多危险？

只要评测信息提前进入训练或方案设计，就可能造成泄漏。常见形式包括：

泄漏类型	例子	为什么危险
训练-测试重复	同一文本/题目同时存在于训练与测试	模型可能直接记忆
标签泄漏	输入特征里偷偷包含答案信息	获得不真实的高分
时间泄漏	用未来信息预测过去	现实部署时拿不到
评测泄漏	反复根据测试结果改系统	测试集变成隐形验证集
Benchmark contamination	公开题目进入预训练/微调数据	难判断是真能力还是见过题

现代 LLM

评测尤其困难，因为预训练数据规模巨大，研究者往往无法完全知道模型是否见过某道公开测试题。

一个实用原则

公开 benchmark 的高分是证据，但不是能力的全部。越是重要的结论，越应该使用多数据集、动态题目、隐藏测试集、人工审计或机制实验来交叉验证。

11. Validation 也会过拟合：超参数搜索的隐藏问题

想象你在同一个验证集上尝试 500 个不同模型。即使每个模型都没有“训练”验证集，你最终选中的那个模型也可能只是偶然最适合这份验证数据。

这就是模型选择偏差 (Model Selection Bias) 。

常见缓解办法：

- 保留一个真正最终使用的测试集；
- 使用交叉验证 (Cross-Validation)，尤其在数据量较小时；
- 减少无原则的超参数“刷榜”；
- 预先注册主要指标、主要实验和停机规则；
- 对重要结果在新的数据集或新的时间段上复验。

与科研方法有关

如果你不断看结果、改方案、再看同一个 benchmark，研究过程本身也在“学习 benchmark”。所以一个可靠论文不能只靠单一榜单，而要靠可复现的现象、机制解释和跨条件验证。

12. 交叉验证 Cross-Validation：什么时候有用？

当数据很少时，一次简单的 70/15/15 划分可能会因为随机性导致结论不稳定。交叉验证的思想是：让不同样本轮流做验证集，然后对结果取平均。

以 5-fold 为例：

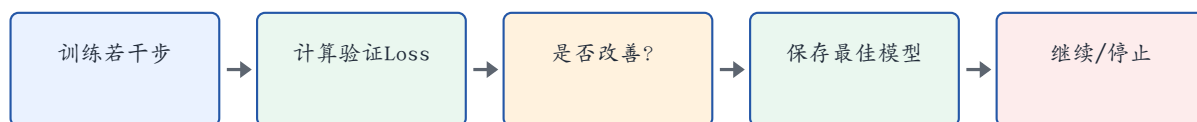
轮次	Fold1	Fold2	Fold3	Fold4	Fold5
1	验证	训练	训练	训练	训练
2	训练	验证	训练	训练	训练
3	训练	训练	验证	训练	训练
4	训练	训练	训练	验证	训练
5	训练	训练	训练	训练	验证

但对超大模型预训练，完整交叉验证往往成本太高。它更常用于传统机器学习、小数据微调、下游任务，或者某些低成本 Agent 实验。

13. 早停 Early

Stopping：一种最直观的抗过拟合方法

训练过程中持续观察验证集 Loss。当训练集 Loss 继续下降，但验证集 Loss 不再改善甚至变差时，可以停止训练，并保存验证集表现最好的 checkpoint。



这背后的思想非常朴素：不要等模型把训练数据的细节学得越来越死，再去部署。

不过对于 LLM 预训练，训练曲线、数据规模和优化策略非常复杂，早停不一定是唯一或主要手段。

其他常见的泛化/正则化手段包括：

- 更多高质量数据
- 数据增强
- Weight Decay
- Dropout（某些架构/阶段）
- 限制模型复杂度
- 合理超参数
- 混合不同来源与任务的数据

14. 语言模型里的“训练集/验证集/测试集”长什么样？

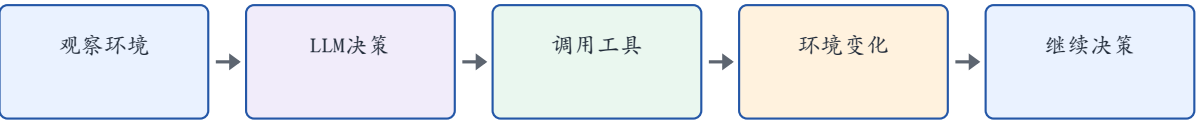
LLM 的数据规模巨大，但基本思想没有变。

阶段	典型数据	主要目的
预训练 Train	网页、书籍、代码、论文等海量 token	学习语言与世界统计规律
预训练 Validation	留出的文本 token	观察 Loss、调训练策略
能力评测 Test/Benchmark	MMLU、代码、数学、长上下文等任务	测量能力边界
对齐/后训练 Train	指令数据、偏好数据、RL 任务	让模型更会遵循任务/偏好
部署评测	真实用户任务、内部隐藏集、红队集	验证实际可靠性

需要注意：LLM 的“benchmark 测试集”往往与预训练数据完全隔离很难做到，所以现代评测越来越强调隐藏题、动态生成、污染检测、时间切分和真实环境任务。

15. Agent Systems 中，为什么泛化问题更复杂？

普通监督学习往往输入一次、输出一次；Agent 则处在环境里连续行动。因此每一步错误都可能改变后续状态。



这会带来几类额外泛化问题：

问题	例子
任务泛化	训练/调试时是订机票，测试时变成退票+改签组合
工具泛化	API 字段变化、返回格式变化、工具临时失败
长程泛化	短任务表现好，但 50 步任务中误差累积
交互泛化	另一个 Agent 的行为发生变化
环境泛化	网页、软件版本、文件结构与 benchmark 不同

与你未来科研直接相关

Agent 的可靠性不能只看“平均成功率”。还需要研究：错误如何累积、失败是否可恢复、换模型/工具/任务后规律是否仍成立、观察到的现象是否跨环境稳定。

16. 一个研究例子：多 Agent 讨论真的更好吗？

假设你研究“多 Agent 讨论是否优于单 Agent”。如果只在一组公开题上不断修改 prompt，最后发现多 Agent 高 5%，这个结论很可能不够强。

更合理的实验结构可能是：

- 训练/开发集：用于调试通信格式、停止规则、角色提示；
- 验证集：选择讨论轮数、Agent 数量、聚合策略；
- 测试集：最终比较单 Agent 与多 Agent；
- 跨模型验证：换不同基础模型，看现象是否仍存在；
- 跨任务验证：推理、知识、代码等任务是否一致；
- 机制指标：除了准确率，还看错误相关性、错误传播、错误纠正率。

科研洞察

真正有研究价值的问题通常不是“某一组参数能不能刷高分”，而是“一个现象是否真实、稳定、可解释，并在不同条件下仍能重复出现”。这就是泛化思想进入科研设计后的样子。

17. 小实验：亲眼观察过拟合

材料包中的 `04_overfitting_demo.py`
使用一个很小的合成数据集，让不同复杂度的多项式去拟合带噪声的数据。

你会看到三种模型：

模型	预期现象
1阶（直线）	太简单，训练和测试都不够好 → 欠拟合
适中阶数	能抓住总体曲线 → 泛化较好
很高阶	训练误差非常低，但测试误差可能升高 → 过拟合

运行：

```
python 04_overfitting_demo.py
```

程序会输出训练 MSE 与测试 MSE，并生成一张 `overfitting_demo.png`。

观察重点

不要只找“训练误差最低”的模型。比较训练误差与测试误差之间的差距，体会“拟合训练数据”和“学到可泛化规律”是两件不同的事。

18. 常见误区

误区1：测试集就是拿来反复调模型的

错。反复用测试结果改方案后，测试集已经失去独立性。

误区2：训练 Loss 越低，模型一定越好

不一定。训练 Loss 低只是说明训练样本拟合好。

误区3：过拟合就是模型参数多

过度简化。数据规模、优化、正则化、任务结构都会影响泛化。

误区4：验证集和测试集完全一样

功能不同。验证集服务于选择，测试集服务于最终客观评估。

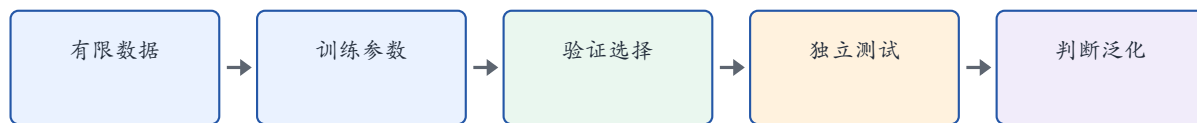
误区5：公开 benchmark 就是真实世界

不是。公开 benchmark 可能有污染、静态化、任务覆盖不足等问题。

误区6：平均分高就说明 Agent 可靠

不够。还应看方差、失败模式、恢复能力、跨条件稳定性。

19. 本讲知识地图



从底层逻辑看，本讲其实只是在回答一个问题：

核心问题

模型在我们手头有限数据上学得很好，是否意味着它在真正没见过的数据、任务和环境上也能工作？

答案不能靠训练分数猜，而需要数据隔离、独立评测、跨条件验证和失败分析。

你以后读 LLM / Agent 论文时，可以用下面四问快速检查实验可信度：

- 训练/开发阶段到底看过哪些数据？
- 验证集负责了哪些选择？
- 测试集是否真正独立，是否可能被污染？
- 结论是否只在一个 benchmark 成立，还是跨模型、任务和环境仍然稳定？

20. 练习题（先自己答，再看答案）

- 1) 为什么不能只用训练集成绩评价一个模型？
- 2) 训练集、验证集、测试集分别承担什么角色？
- 3) 如果你每天根据测试集结果修改 prompt，这个测试集还公平吗？为什么？
- 4) “训练误差很低，验证误差很高”通常提示什么？
- 5) 欠拟合和过拟合的典型表现有什么区别？
- 6) 什么叫泛化？IID 泛化与 OOD 泛化有什么差别？
- 7) 为什么公开 LLM benchmark 存在 contamination 风险？
- 8) 验证集会不会也被“过拟合”？请解释。
- 9) 对于 Agent，为什么真实环境泛化比单轮分类更难？
- 10) 如果你研究多 Agent 协作，一个更可信的实验设计至少应包含哪些数据/条件隔离？

21. 练习题答案

- 1) 训练集参与了参数优化，模型可能记住训练样本；真正关心的是未见数据上的表现。
- 2) 训练集用于更新参数；验证集用于选择模型/超参数/停止点；测试集用于最终独立评估。
- 3) 不公平。你的人工修改把测试信息反馈进了系统，它已经变成事实上的验证集。
- 4) 典型地提示过拟合，也可能提示训练/验证分布差异或数据处理不一致。
- 5) 欠拟合：训练和验证都差；过拟合：训练很好，但验证/测试明显变差。
- 6) 泛化是对未见数据仍能工作。IID 是新样本分布与训练近似一致；OOD 是分布明显变化。
- 7) 公开题目可能进入预训练/微调数据，或者研究社区反复根据该 benchmark 调系统，难以区分真实能力与见题。
- 8) 会。反复根据验证集选择大量方案，相当于逐渐针对这份验证集优化。
- 9) Agent 连续行动会改变环境，错误会累积；工具、网页、其他 Agent、长上下文也会变化。
- 10) 至少分开发/验证/最终测试，并在跨任务、跨模型或跨环境条件下复验关键现象；重要时还要分析机制和失败模式。

22. 本讲最低闭环

如果你只记住 5 件事

① 训练集更新参数；② 验证集负责选择；③ 测试集负责最终独立评估；④ 过拟合 = 训练好但新数据差；⑤ 真正目标不是“背训练题”，而是泛化到未见条件。

下一讲，我们进入 Transformer 出现之前的历史问题：为什么早期序列模型 RNN/LSTM 会遇到瓶颈，以及 Attention 为什么成为关键突破。

下一讲：第7讲《从 RNN、LSTM 到 Attention：Transformer 为什么会出现？》